file, line by line, until the EOF is reached. Every time a line is read, the *numberOfLines* variable is increased by one.

- Line 42: The *defer* statement is used to postpone the execution of a function or a method until just before the enclosing function or method finishes and returns. If there are any return values, they are evaluated before the *defer* statement. The most common use of *defer* is the one presented here: to make sure that a successfully opened file is closed when you do not need it any longer.

A more sophisticated version of the *countLines* program –that could replicate the functionality of the *wc (man wc)* command line utility– is easy to program and is left as an exercise for you to try out.

A more advanced Go example

This part of the article will solve a Quadratic Equation ($ax^2 + bx + c = 0$) using a Web interface. The presented implementation does not solve a Quadratic equation when the *discriminant* is less than zero. The *net/http* standard Go library that is used, makes it exceptionally easy to create HTTP servers. The source code is as follows:

```
// Programmer: Mihalis Tsoukalos
// Date: Wednesday 27 May 2013
//
// Filename: quad.go
//
// This Go program solves Quadratic Equations
// using a Web Interface.
// A Quadratic Equation has the form ax^2 + bx + c = 0
// Note: Does not support solutions with complex numbers.

package main

import (
    "fmt"
    "net/http"
    "log"
    "math"
    "strconv"
)

const (
    decimals     = 2
    pageTop      = `<!DOCTYPE HTML><html><head>
<style>.error{color:#FF0000;}</style></head>
<title>Solving Quadratic Equations</title><body>
<h3>Quadratic Equation Solver for OSFY Magazine</h3><p>Solves equations of the form
a<i>x</i>² + b<i>x</i> + c = 0</p>`
    form         = `<form action="/" method="POST">
<input type="text" name="a" size="1"><label for="a"><i>x</i>²</label> +
<input type="text" name="b" size="1"><label for="b"><i>x</i></label> +
<input type="text" name="c" size="1"><label for="c">=</label>
<input type="submit" name="calculate" value="Solve!">
</form>`
    pageBottom   = "</body></html>"
    error        = `<p class="error">%s</p>`
    solution     = "<p>%s Solution(s): %s</p>"
    oneSolution  = "<i>x</i>=%s"
    twoSolutions = "<i>x</i>=%s or <i>x</i>=%s"
    noSolution   = "<i>impossible to solve! No solutions.</i>"
)

func main() {
        http.HandleFunc("/", solveQuad)
    if err := http.ListenAndServe(":8080", nil); err != nil {
        log.Fatal("Exiting: Cannot start server: ", err)
    }
}

func solveQuad(writer http.ResponseWriter, request *http.Request) {
        // Must be called before writing response
    err := request.ParseForm()
    fmt.Fprint(writer, pageTop, form)
    if err != nil {
        fmt.Fprintf(writer, error, err)
    } else {
        if floats, message, ok := processRequest(request); ok {
            question := formatQuestion(request.Form)
            x1, x2 := solve(floats)
            answer := formatSolutions(x1, x2)
            fmt.Fprintf(writer, solution, question, answer)
        } else if message != "" {
            fmt.Fprintf(writer, error, message)
        }
    }
    fmt.Fprint(writer, pageBottom)
}

func formatQuestion(form map[string][]string) string {
    result := formatSignAndNumber("", form["a"][0], "<i>x</i>²")
    result += formatSignAndNumber(" ", form["b"][0], "<i>x</i>")
```